

重点回答 | SGLang 与 vLLM 的 KV Cache 机制

基于 SGLang (NeurIPS 2024)、vLLM (SOSP 2023) 论文及官方文档。

1. RadixAttention / RadixCache 解决什么问题？

多个请求若共享系统提示词或历史对话前缀，重复执行 Prefill 会重复计算相同 token 的 KV。SGLang 将已计算的 KV 按 token 前缀组织成基数树 (RadixCache)；新请求查找最长公共前缀，复用命中的 KV，只计算后续未命中部分。完成后可把新前缀插入树，空间不足时按策略淘汰未使用的叶节点。核心收益是减少跨请求的重复计算和首字延迟。

2. 分页 KV 与前缀复用在推理流程的哪里？

请求进入服务后先分词并排队；调度 / Prefill 前后进行前缀匹配，命中的前缀 KV 直接接入本次请求，Prefill 仅处理剩余 token。分页 KV 是底层存储与寻址方式：Prefill 写入 K/V，Decode 每生成一个 token 就读取历史 KV 并追加新的 K/V；逻辑连续的序列可映射到非连续的物理页。前缀复用决定“哪些 KV 不必重算”，分页管理决定“这些 KV 如何放置、读取和扩展”。SGLang 默认 page size 为 1 个 token，也支持配置。

3. vLLM 的 PagedAttention 解决什么问题？

生成长度事先未知，若为每个请求预留连续的大块 KV 空间，容易产生预留浪费、内外碎片和重复副本，限制同时运行的请求数。PagedAttention 把 KV 划为固定大小的块，由块表把逻辑位置映射到非连续的物理块；按需分配，并可在适用场景共享块。因此能提高 KV 内存利用率，支持更大的 batch / 吞吐量。

4. 两者是什么关系，有何不同？

二者互补，不是互相替代：PagedAttention 主要解决 KV 的分页存储、寻址与内存利用；RadixAttention 主要解决多请求之间“哪些相同前缀值得保留、匹配和复用”，并结合调度减少 Prefill。SGLang 的树结构按可变长度前缀共享 KV；vLLM 当前也有自动前缀缓存 (APC)，使用基于哈希的完整块匹配，所以说 vLLM 完全不支持前缀复用。比较原始论文时，应区分 PagedAttention 本身与后续加入的 APC。

参考资料

- [1] Zheng et al., SGLang, NeurIPS 2024. papers.nips.cc/paper_files/paper/2024/file/724be4472168f31ba1c9ac630f15dec8-Paper-Conference.pdf
- [2] Kwon et al., Efficient Memory Management for Large Language Model Serving with PagedAttention, SOSP 2023. arxiv.org/pdf/2309.06180
- [3] vLLM Docs, Automatic Prefix Caching. docs.vllm.ai/en/latest/design/prefix_caching/; SGLang Docs, Server Arguments. docs.sglang.io/docs/advanced_features/server_arguments